SW Engineering CSC 648/848 -Spring 2026 Section 02

Team 01, Milestone 2

03/10/2026

Project Name: LifeTrack

Team Members
Khaterina Sengchareune - Team lead & Github Master
ksengchareune@sfsu.edu

Mohit Kumar - Database Engineer
mkumar3@sfsu.edu

John Narnaja - Backend Engineer
jnaranja@sfsu.edu

Alex Hernandez - Backend Engineer
ahernandez75@sfsu.edu

Will Brust - Frontend Engineer
wbrust@sfsu.edu

Lasiru Weerasuriya - Frontend Engineer
lweerasuriya@sfsu.edu

Revisions Made	Date submitted	Date Revised

1. <u>Functional Requirements - prioritized</u>

Rating: 1- Must have, 2- desired, 3 - opportunistic

1. **A powerful search and discovery engine:** (Rating: 1) Users will be able to find and explore content on the site.
   1.1. There will be a single search bar across all pages that users can use to look for all types of content.
   1.2. Search results will be displayed in a separate page with clear titles, short descriptions, and icons.
   1.3. A “Discover” section on the public homepage will have a rotating selection of featured content without requiring users to search for it.
2. **User Account Registration and Login**: (Rating: 1) New users will be able to create an account, log in, and log out to access personalized features like dashboards and saved content.
   2.1. Users can register a new account with a unique username and password, as well as a set of security questions.
   2.2. Standard username/password authentication will be required to log into an account.
   2.3. Users will be able to recover forgotten passwords by answering a set of security questions.
3. **User Roles and Permissions**: (Rating: 1) This web application will have different access privileges for guest users, student users, faculty and academic program users, university support organization users, and admins.
   3.1. Guest users only have read-only access to the “Discover” section and public community content.
   3.2. Students have full access to create and manage their own dashboards, goals, projects, milestones, and reflections. They can also save public content to their personal library.
   3.3. School faculty and university organizations will have a special content publisher view. They can create, publish, and update academic resources but cannot see or interact with a student’s personal dashboard data.
   3.4. Admins shall have the ability to manage all user accounts and platform content.
4. **Personal Dashboard**: (Rating: 1) Students will have the ability to create and use a personal dashboard to view and manage goals, projects, milestones, reflections, saved items, and progress summaries.
   4.1. The Dashboard Home View is a central hub for displaying content to the user, which will offer various features, including the ability to view upcoming milestones, daily schedules, and a course overview.
5. **Goal Management**: (Rating: 1) Student users will be able to create, view, edit, and delete goals and track goal status over time.
   5.1. Students can enter a separate “Goals” page to create and manage goals. Students can Create new goals with a title, short description, and various tags and modifiers.
6. **Project Management**: (Rating: 1) Students will be able to create and manage projects that organize goals and activities into a structured plan.
   6.1. Students can enter a separate “Projects” page where user-created projects are made and listed. Students can create projects with a name, short description, and various tags and modifiers. Students can also add goals to projects.

7. **Milestone management**: (Rating: 1) Students will be able to add milestones to projects and update milestone progress over time.

   7.1. Milestones can have tags and attributes to help organize them, including a name, short description, and due date.

8. **Accessibility**: (Rating: 1) This web application will be made available and usable to as many people as possible, as well as offering various settings to enhance the user experience.
9. **Intelligent search**: (Rating: 2) Users can use filters to narrow down and enhance their searches. 9.1. Users can interact with a set of filters to narrow the number of entries when using the search bar. There will be many options for filters, including filtering by certain tags or keywords.

10. **Reflection and journaling:** (Rating: 2) Users will be able to write reflections weekly or on demand to document progress, learning, and challenges.

    10.1. Users will be able to choose a set of prompts or write freely for their reflections. Users will also be able to look at past reflections.

11. **Save and organize discovered content:** (Rating: 2) Registered users will be able to save resources, templates, workflows, and opportunities for later use and organize them inside their dashboard.

    11.1. Users will have access to a separate webpage that will contain all of their saved content.

12. **In-app notifications:** (Rating: 3) Users will receive in-app notifications for important events.

    12.1. Notifications will include upcoming milestone due dates, upcoming project due dates, and updates from the admins. Users will also be able to delete or close notifications.

13. **Archive for templates:** (Rating: 3) There will be an archive of simple templates for workflow and goals, giving users access to some beginner tools that can be modified.

    13.1. A public "Template Library" page will be accessible from the main navigation menu, where it holds simple text-based guides, pre-filled structures, and templates for use by all users. These templates will be created and offered by the admins.

Modified Functional Requirements

1. **In-app messaging and notifications → In-app notifications**
    a. The implementation of a real-time, in-app messaging system is incredibly complex and can take up a majority of time and resources. Real-time messaging would require its own database tables, separate messaging logic, and a separate user interface for an inbox and conversations. Instead, users will now get notifications in-app for certain things such as password resets and newly uploaded/updated content sent by the Admins. Additionally, since no external e-mail clients are not allowed, all notifications are sent in-app.
2. **User Account Registration and Login**
    a. Due to restrictions preventing the use of external e-mail clients, users only need to provide a unique username and a password to create an account. There will also be a set

of personal security questions that users can choose from in a dropdown menu, as an added layer of security.

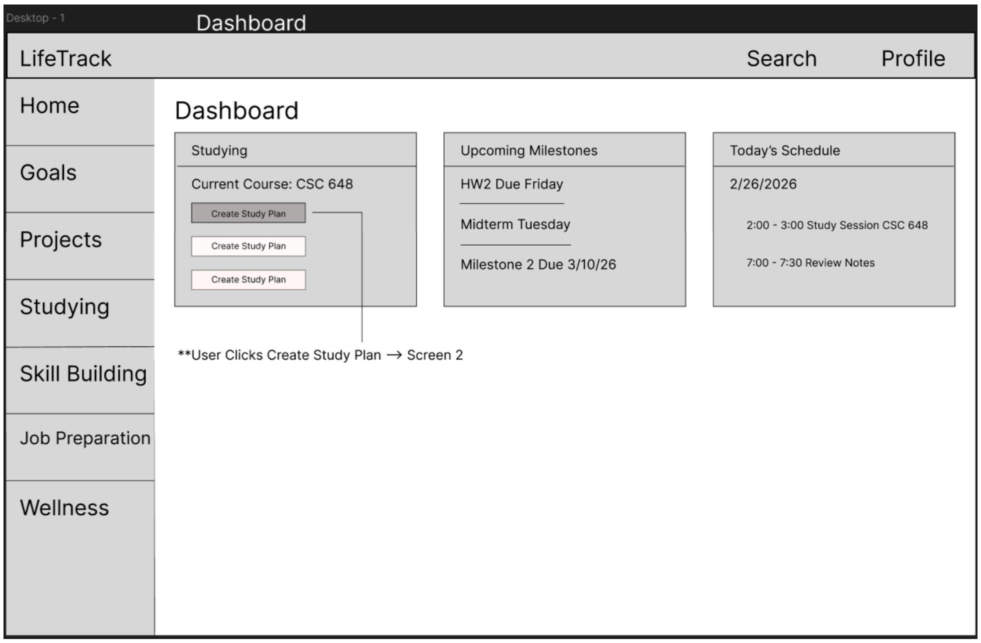
<u>**Removed Functional Requirements**</u>

1. **AI Workshops**
    a. Due to the optionality of AI feature implementation to our web application, it was decided that the functional requirement, **AI Workshops**, was to be removed due to no longer being in our team's scope. Additionally, some of the non-AI features of the **AI Workshops** functional requirement can be taken over and grouped into other key features, thus reducing redundancy.
2. **Document user time and check in on the user**
    a. Implementing accurate time tracking for various user activities (or inactivity) requires significant logic to determine active time versus idle time. An advanced understanding of server-side scheduling, client-server synchronization, and background scripts is also required to fully implement this function. Additionally, extra storage and space in the database need to be set aside to keep track of all a user's start and stop times, which could lead to poor space optimization for a smaller, simpler database design.
3. **Resource publishing for campus groups**
    a. The functional requirement "User Role and Permissions" already handles publishing and permissions for students and faculty. A separate data table will need to exist to handle groups, memberships, and roles. Additionally, this would require additional interfaces for group management that are separate from a user's personal dashboard, doubling the UI work required from the frontend and the logic required from the backend.
4. **Interactive Dashboard**
    a. Removed because a prior existing functional requirement, **Personal Dashboard**, already encompasses the features that **Interactive Dashboard** has listed in its high-level description.
5. **Exporting of Documents**
    a. Removed for redundancy due to an existing functional requirement already encompassing its features and more.
6. **Personalization and discovery of new content**
    a. Implementing a recommendation engine requires some complex logic in order to keep track of user behavior and analyze certain patterns to serve dynamic content. Instead, the "Discovery" widget on the Dashboard serves as a simple "recommendation system" that is curated by the Admins to showcase featured content.
7. **Importing of the user's personal university resources**
    a. Certain document types, such as PDFs and DOCX, are relatively complex to import and transfer to a web application, requiring extra libraries and advanced logic to handle layout and data parsing. In addition, official academic websites and resources already host a student's record in their database, making our implementation redundant and too simple to handle all of a student's needs.

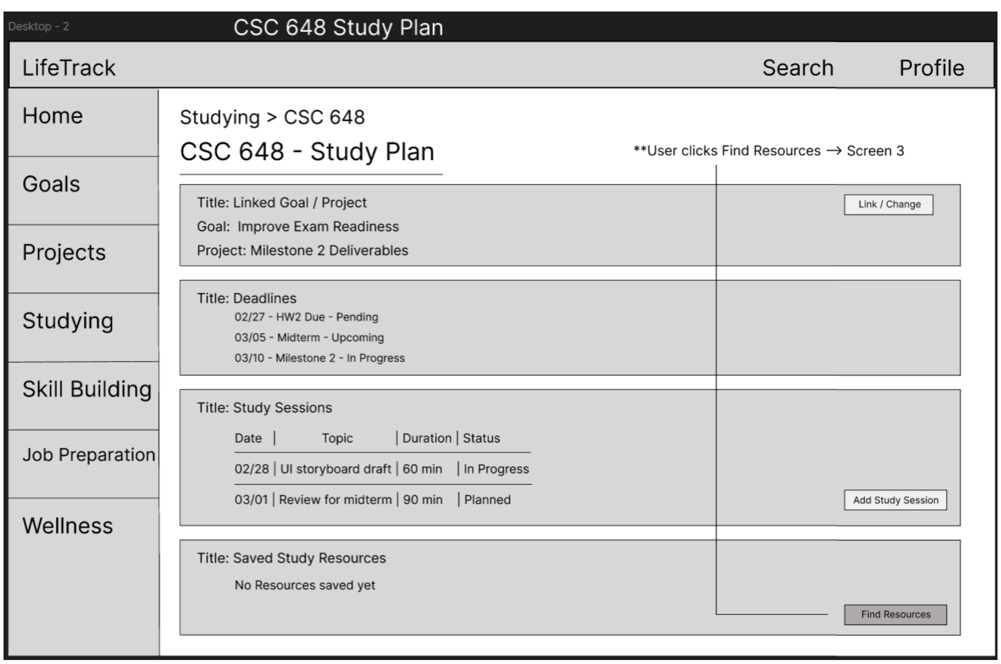
2. UI Mockups & Storyboards (high level only)

Use Case 1: Studying

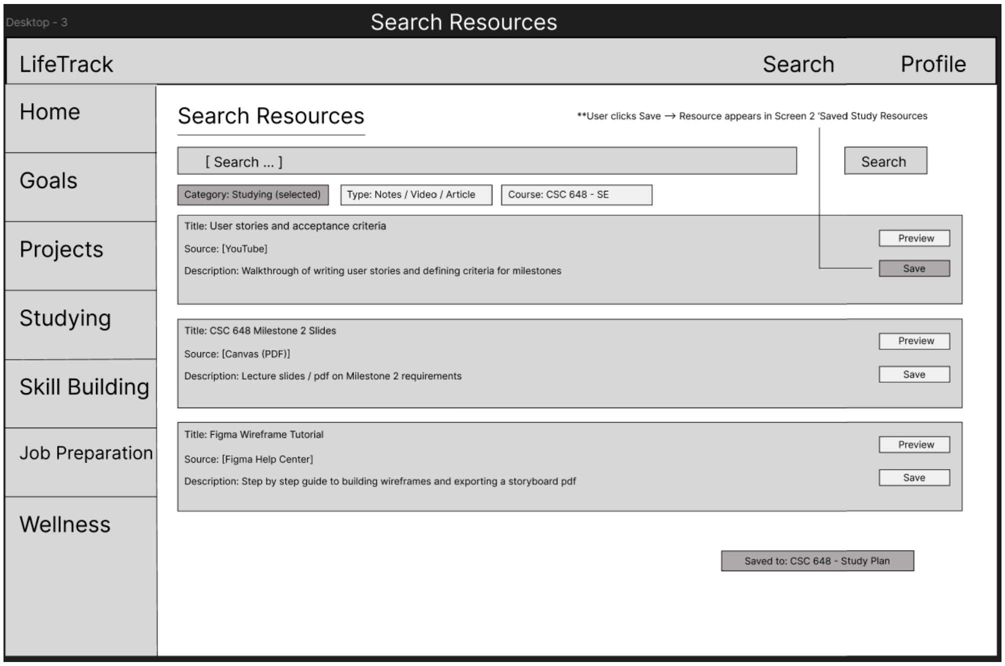
1. **Dashboard**: The user opens LifeTrack, checks upcoming deadlines and today's study schedule, then selects **Create Study Plan** under the Studying section to begin organizing their work for a course.



1.2. Study Plan / Course Workspace: The user links the plan to a relevant goal/project, reviews upcoming milestones (deadlines), adds or views planned study sessions, and chooses **Find Resources** to support their study plan.

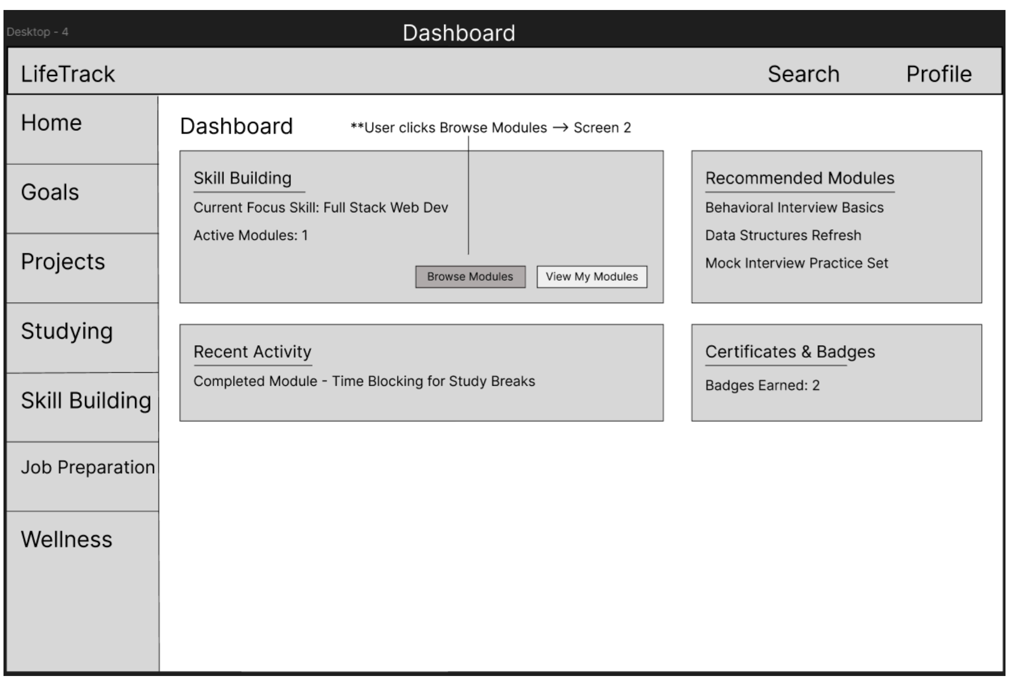


1.3 Search Resources: The user searches and filters results to **study** for their course, previews a resource if needed, then clicks **Save** to attach it to the CS 648 study plan, so it appears later under **Saved Study Resources** in the course workspace.

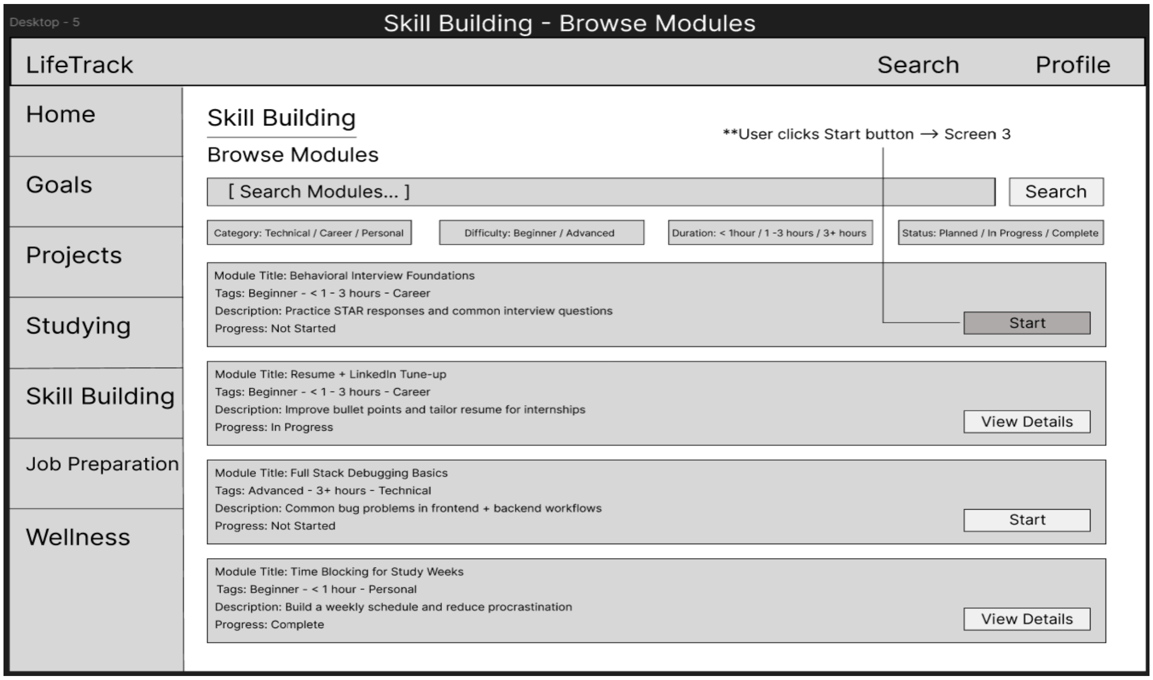


Use Case 2: Skill Building

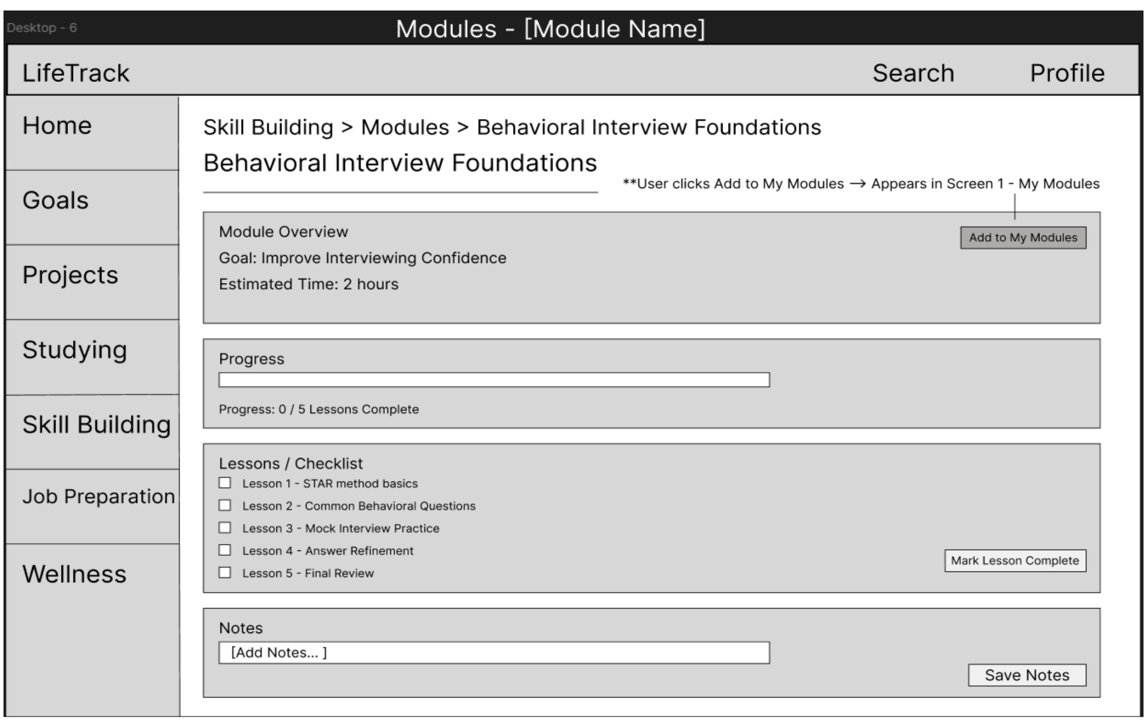
2. Dashboard: The user starts skill-building by choosing to browse modules from the dashboard.



2.2 Browse Modules: The user searches/filters modules and selects one to start.

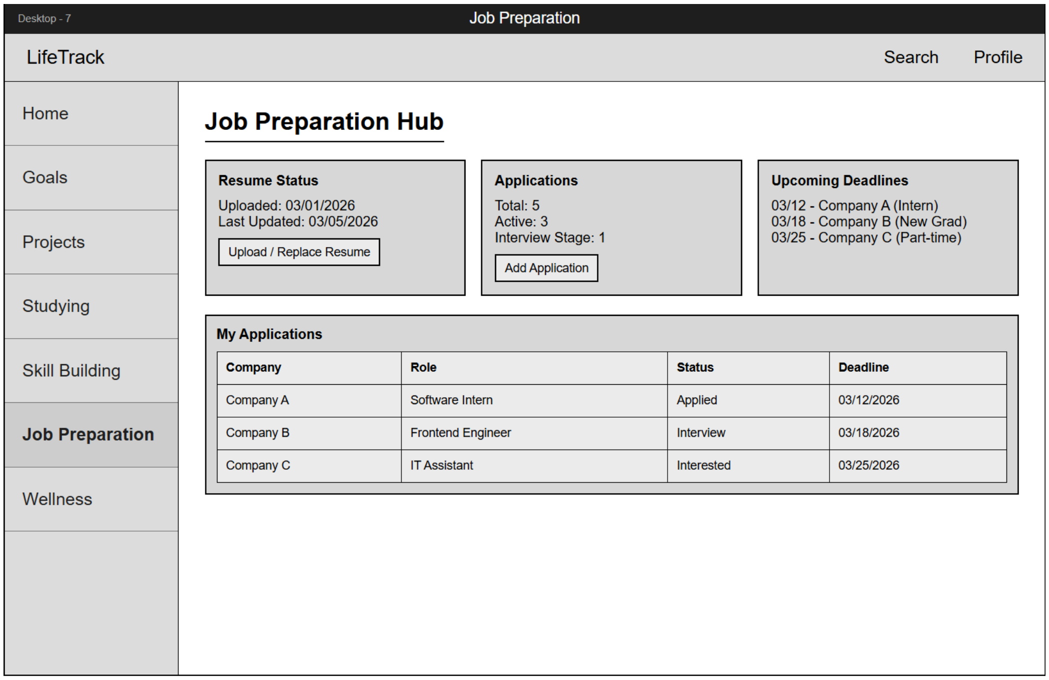


2.3 Module Tracker: The user adds the module and can complete a lesson, which will update progress (reflected back on the dashboard).

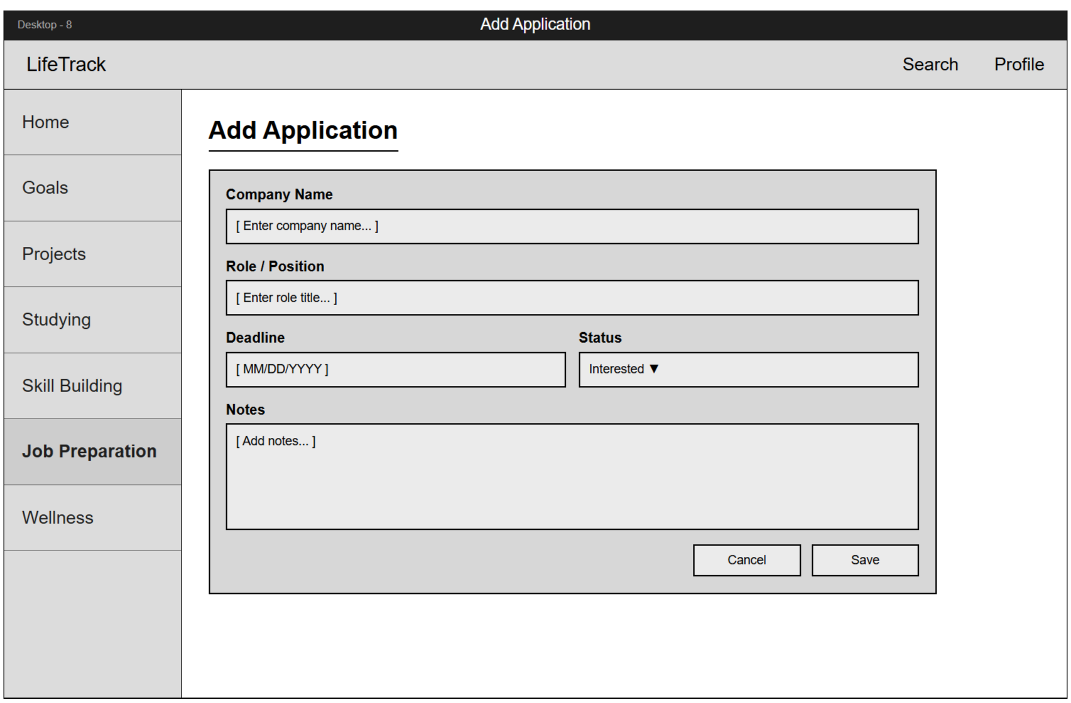


Use Case 3: Job Preparation

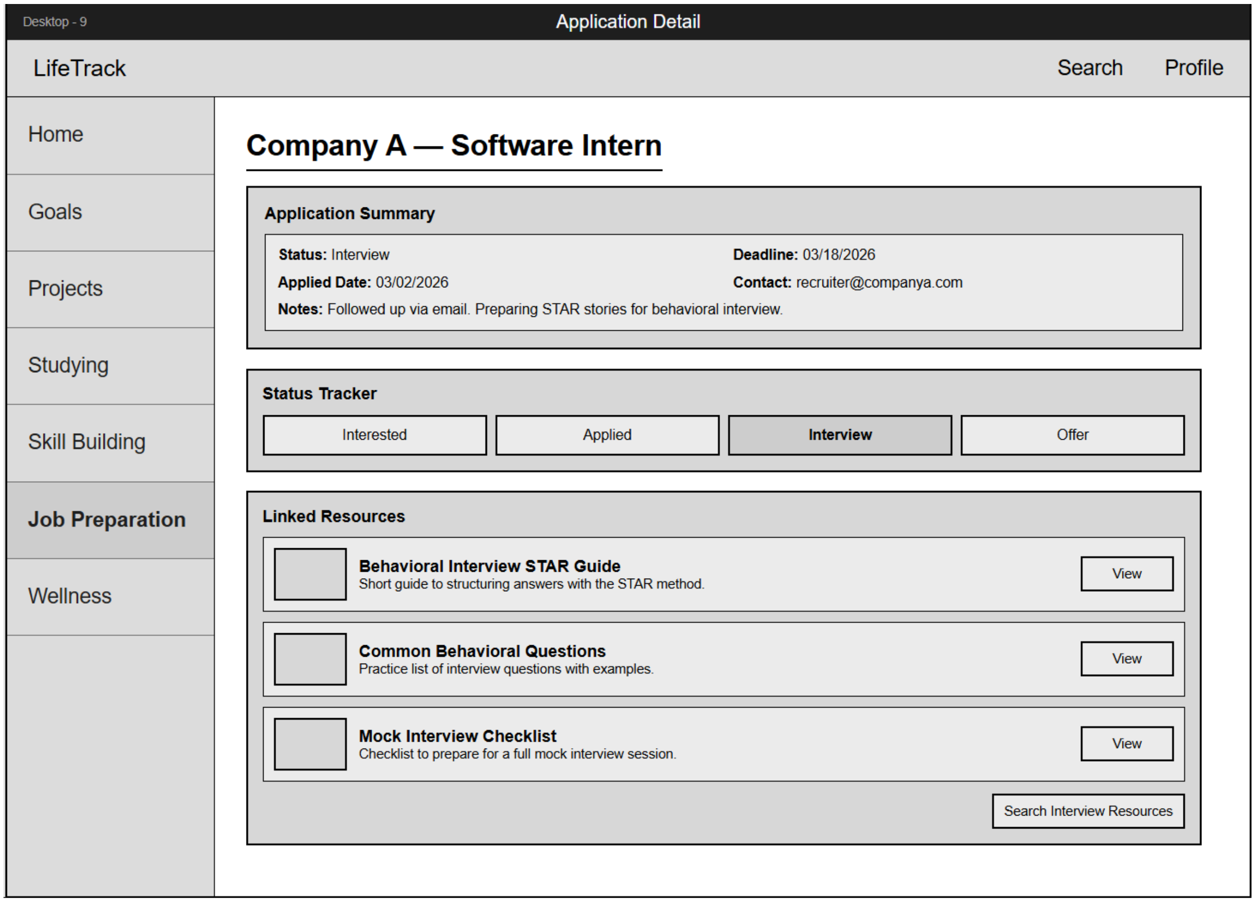
3.1 Job Preparation Hub: The student views an overview of job application activity, including resume status, total applications, and upcoming deadlines. and can initiate a new application entry.



3.2 Add Application: The student records a new job or internship opportunity by entering company details, deadline, status, and notes to track progress.

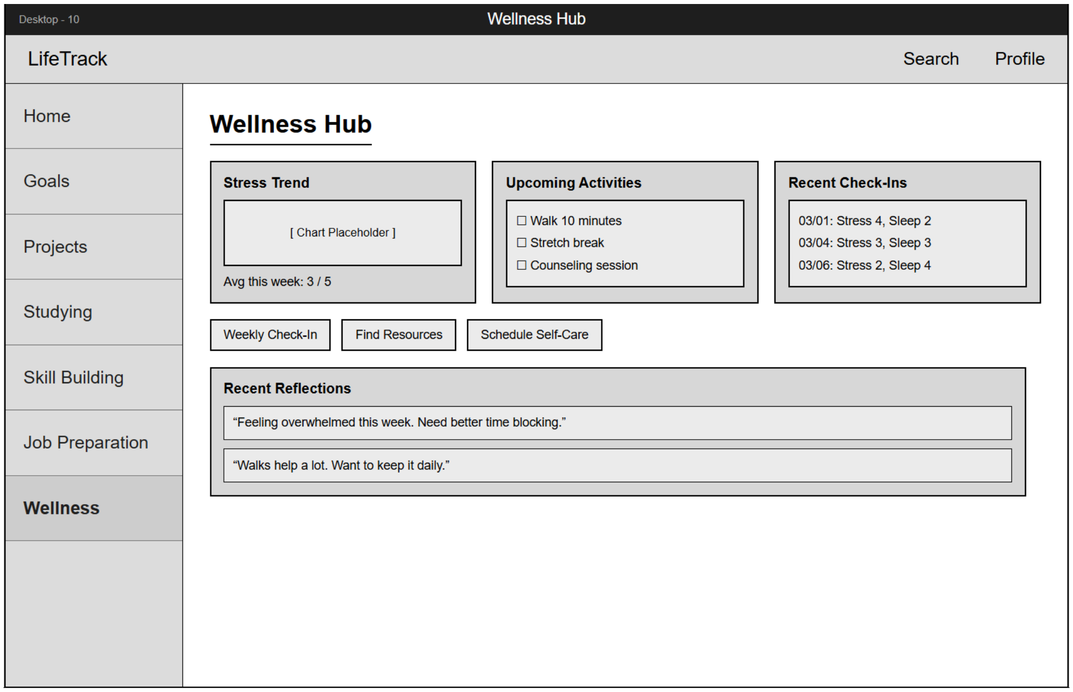


3.3 Application Detail: The student reviews detailed application information, monitors status progression, and accesses related interview preparation resources

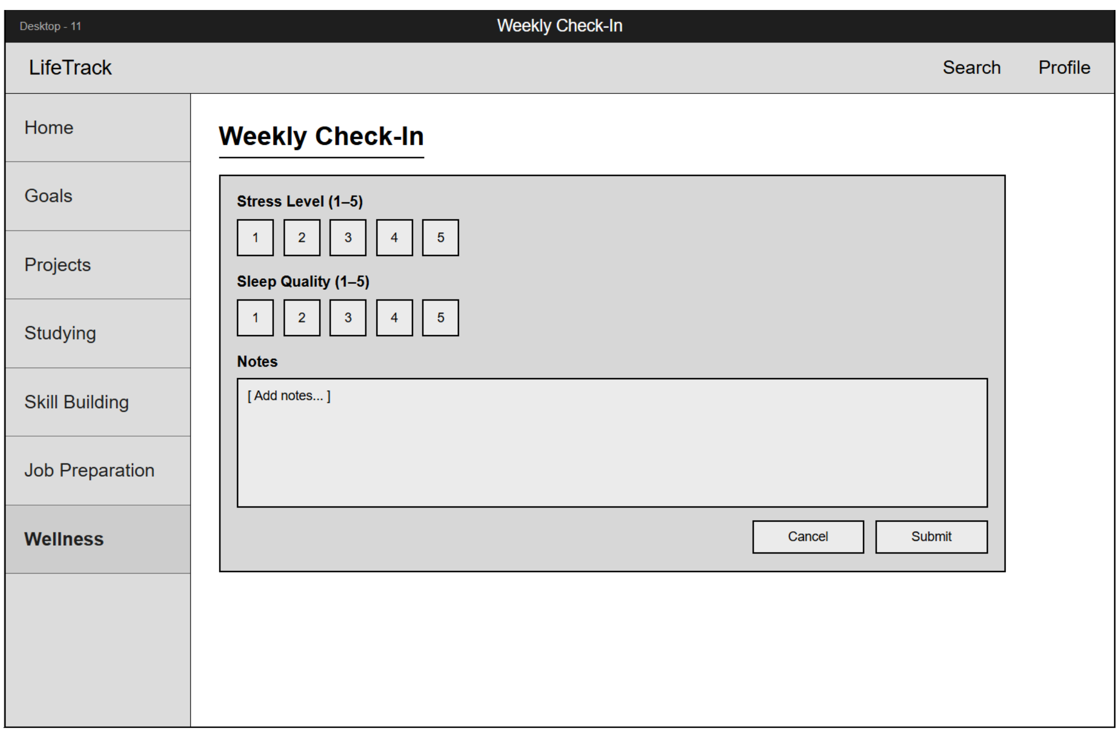


Use Case 4: Mental Wellness

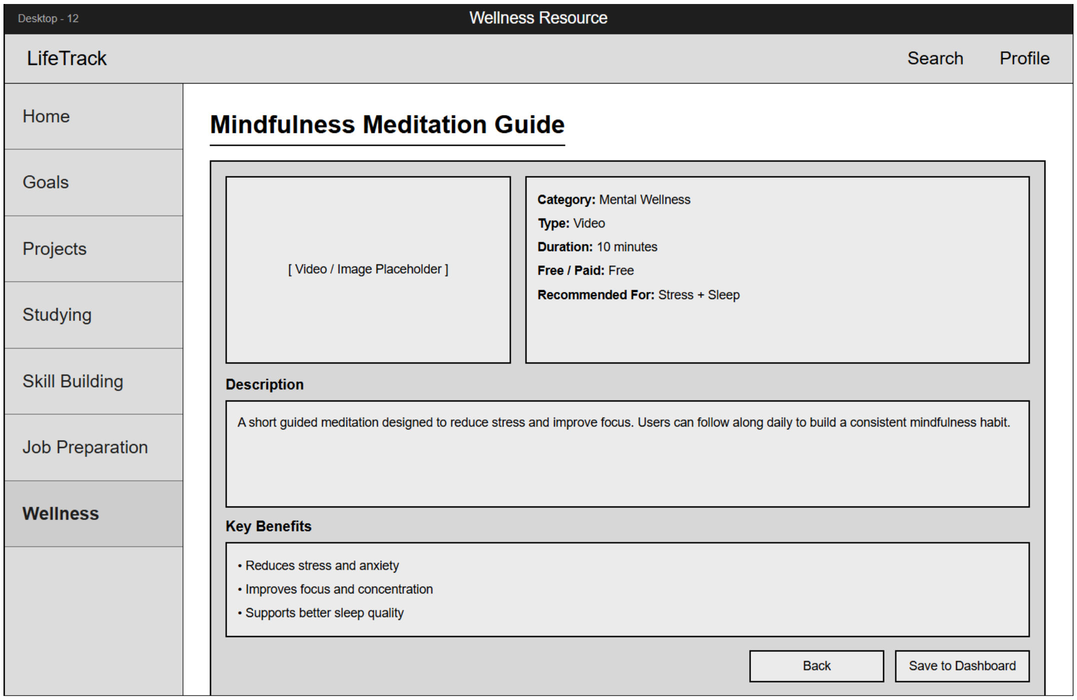
4.1 Wellness Hub: The student views stress trends, upcoming self-care activities, and recent reflections, and can begin a weekly check-in.



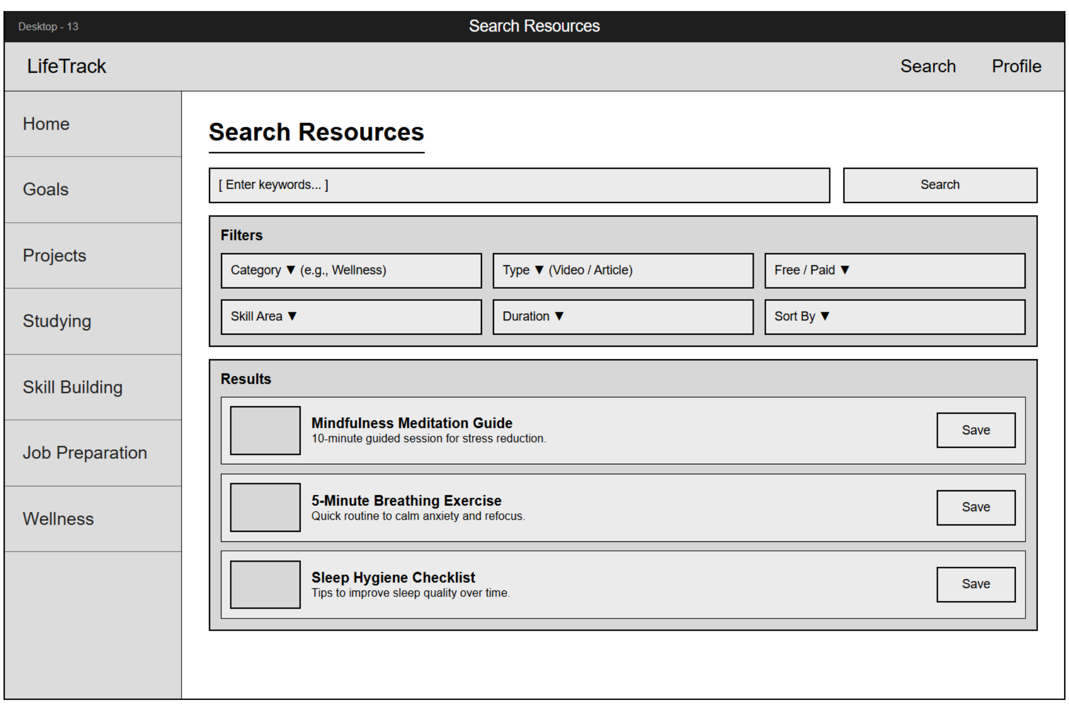
4.2 Weekly Check-In: The student completes a structured wellness check-in by rating stress level and sleep quality and recording personal reflections.



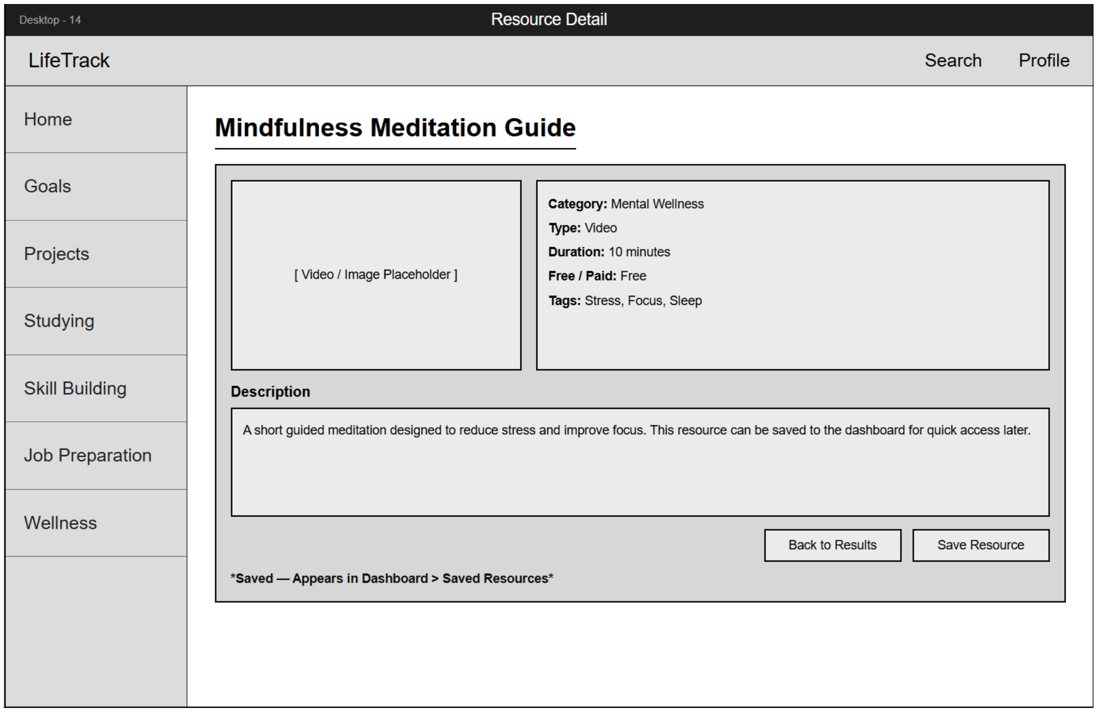
4.3 Wellness Resource Detail: The student reviews detailed information about a selected wellness resource and can save it to their dashboard.



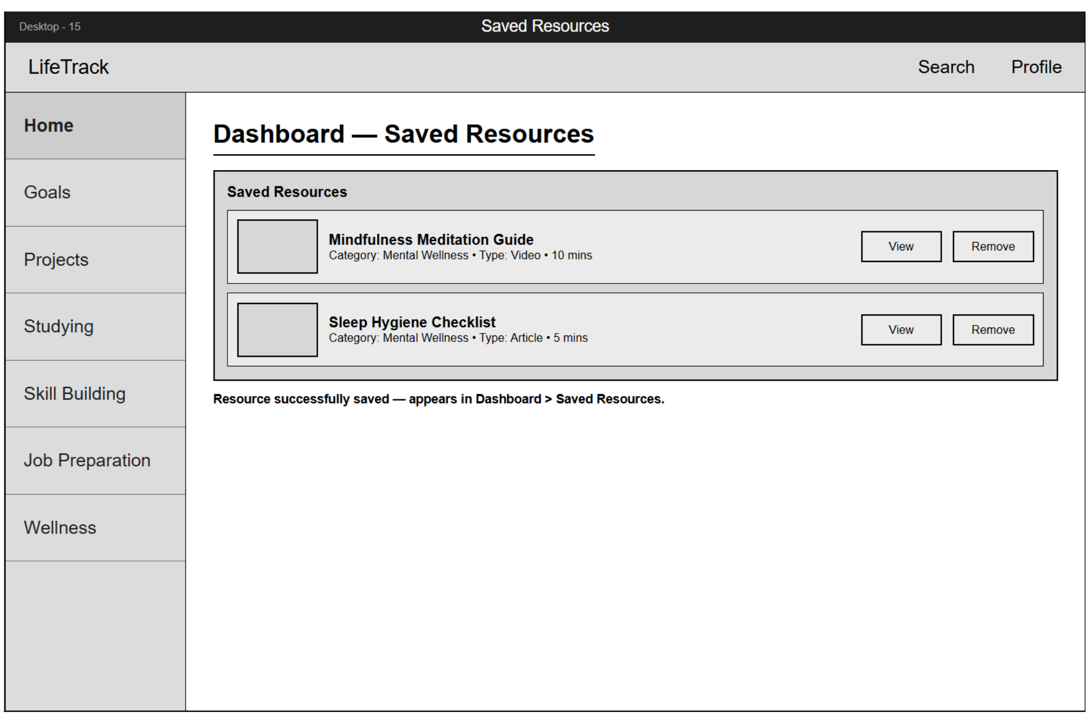
4.4 Search Resources: The student searches for wellness-related resources using filters and keyword search before selecting items to save.



4.5 Resource Detail Confirmation: After saving a resource, the system confirms persistence and provides navigation options back to search or the dashboard.



4.6 Dashboard-Saved Resources: The dashboard displays all saved resources, confirming that selected items persist within the user's saved collection.



3. High-level Architecture, Database Organization

High-level architecture

LifeTrack uses a simple 3-tier web application architecture.

Client

- Users access the app in a web browser.
- Guest users can browse public content and use search and filters.
- College student users can access features like the personal dashboard, goal management, project management, milestone management, reflections, and saved items.

Server

- A Node.js and Express backend handles authentication, roles, and permissions, and application logic.
- The backend exposes REST endpoints so the browser can safely read and update data.
- The backend returns JSON responses to the client.

Database

- MySQL stores persistent data for accounts, dashboards, goals, projects, milestones, reflections, resources, and saved items.
- The backend is the only component that connects to MySQL.

Deployment

- Nginx serves the frontend and reverse proxies API requests to the backend.
- MySQL runs on the same VM for a simple setup.

DB organization

The database schema is organized around the features and names in Section 1.

User account registration, login, roles, and permissions

user_accounts

- Purpose
    - Stores login identity and account status for registered users.

- Fields
    - account_id
    - email
    - username
    - password_hash
    - account_status
    - created_at

user_security_questions

- Purpose
  - Stores the security questions and hashed answers used to verify a user when they forgot their password.
- Fields
  - question_id
  - account_id
  - question_text
  - answer_hash
  - created_at
  - updated_at

roles

- Purpose
  - Stores the role names used in the system.

- Fields
  - role_id
  - role_name
  - example values are guest, college_student, faculty_academic_program, university_support_organization, administrator

user_roles

- Purpose
  - Links accounts to roles so permissions can be enforced.
- Fields
  - account_id
  - role_id

<u>Accessibility</u>

user_accessibility_settings

- Purpose
  - Stores accessibility preferences so the interface can adapt per user.

- Fields
  - settings_id
  - account_id
  - theme_mode
  - text_size
  - high_contrast_enabled
  - font_choice
  - color_blind_mode

Personal dashboard and planning

dashboards

- Purpose
  - Stores the personal dashboard workspace for a college student user.
- Fields
  - dashboard_id
  - account_id
  - last_viewed_at

goals

- Purpose
  - Stores goals for goal management.
- Fields
  - goal_id
  - account_id
  - title
  - description
  - status
  - target_date nullable
  - notes nullable
  - category nullable
  - created_at
  - updated_at

projects

- Purpose
  - Stores projects for project management.
- Fields
  - project_id
  - account_id
  - title
  - description
  - status
  - created_at
  - updated_at

project_goals

- Purpose
  - Links goals to projects so goals can be grouped into a plan.
- Fields
  - project_id
  - goal_id

milestones

- Purpose

    - Stores milestones inside projects for milestone management.
- Fields
    - milestone_id
    - project_id
    - title
    - description
    - status
    - due_date nullable
    - sort_order nullable
    - created_at
    - updated_at

<u>Reflections and journaling</u>

reflections

- Purpose
    - Stores reflections and supports linking reflections to goals, projects, milestones, and resources.
- Fields
    - reflection_id
    - account_id
    - title nullable
    - body_text
    - created_at
    - updated_at

reflection_links

- Purpose
    - Connects a reflection to goals, projects, milestones, and resources.
- Fields
    - reflection_id
    - linked_type
    - linked_id
    - linked_type values are goal, project, milestone, resource

<u>Search and discovery content</u>

resources

- Purpose
    - Stores discoverable content items like academic and career resources, improvement tools and learning aids, campus opportunities and services, and public community shared content.
    - Modules are stored as resources where content_type equals module.
    - Templates are stored as resources where content_type equals template.
    - Administrators can moderate public content by setting is_public to false.
- Fields
    - resource_id
    - title

- description
- url nullable
- content_type
- category
- use_case nullable
- skill_area nullable
- is_non_ai
- is_free
- is_public
- created_by_account_id nullable
- created_at
- updated_at

resource_tags

- Purpose
  - Tags support flexible filters like studying, job preparation, productivity, and other custom labels beyond the fixed fields.
- Fields
  - resource_id
  - tag

<u>Save and organize discovered content</u>

saved_items

- Purpose
  - Stores the relationship between a college student user and a resource they saved.
  - Supports collections and attachments to goals, projects, or milestones.
- Fields
  - saved_item_id
  - account_id
  - resource_id
  - collection_name nullable
  - linked_type nullable
  - linked_id nullable
  - linked_type values are goal, project, milestone

notifications

- Purpose
  - Stores in-app alerts for users, such as upcoming milestone due dates and important updates, so they can review and clear them inside the platform.
- Fields
  - notification_id
  - account_id
  - notification_type

- message_text
- is_read
- created_at

Key relationships

- One user_account can have one dashboard and one user_accessibility_settings record.
- One user_account can have many goals, projects, reflections, and saved_items.
- One project can have many milestones.
- Reflections can link to goals, projects, milestones, and resources through reflection_links.
- Resources support discovery filters through fields like category, use_case, skill_area, is_non_ai, and is_free.

Media storage

Images and video or audio will be stored in the file system, not in MySQL BLOBs.

- The database stores only a file path if media is stored on the server, or a URL if it is external.

Special data formats

- No special formats are required.

Search and filter architecture and implementation

This describes how search works for users and how it is implemented in the system.

User experience

- Users type normal keywords into the search bar.
- Users apply filters using UI controls for category, use case, content type, skill area, non-AI resources, and free vs paid.
- The app shows results as a list of content cards.

Search software and algorithm

- Search is implemented using MySQL queries from the backend.
- Keyword matching uses SQL LIKE for simple contains matching.
- Filters use exact matches on fields and optional tag matching through resource_tags.

Database terms searched

- resources.title
- resources.description
- resources.category
- resources.use_case
- resources.skill_area
- resource_tags.tag

How search items are organized for the user

- Results are grouped as discoverable content cards, including tools, templates, workflows, modules, resources, opportunities, services, and public community shared content.
- Default sorting is newest first using created_at.
- If relevance sorting is enabled, a simple scoring rule is used where title matches are ranked above description matches.

Our own APIs

These are major backend endpoints our team will create using Express. They map directly to the functional requirements and follow a simple CRUD(create, read, update, delete) pattern.

Authentication and user accounts

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout
- POST /api/auth/verify-security-answers
- POST /api/auth/set-new-password
- GET /api/auth/me

Roles and permissions

- GET /api/users/me/roles

Personal dashboard

- GET /api/dashboard

Goals

- GET /api/goals
- POST /api/goals
- PUT /api/goals/{goal_id}
- DELETE /api/goals/{goal_id}

Projects

- GET /api/projects
- POST /api/projects
- PUT /api/projects/{project_id}
- DELETE /api/projects/{project_id}

Milestones

- GET /api/projects/{project_id}/milestones
- POST /api/projects/{project_id}/milestones
- PUT /api/milestones/{milestone_id}
- DELETE /api/milestones/{milestone_id}

Reflections

- GET /api/reflections
- POST /api/reflections
- PUT /api/reflections/{reflection_id}
- DELETE /api/reflections/{reflection_id}

Resources and discovery search

- GET /api/resources
- GET /api/resources/{resource_id}
- GET /api/resources/search

Saved items

- GET /api/saved-items
- POST /api/saved-items
- DELETE /api/saved-items/{saved_item_id}

Accessibility settings

- GET /api/accessibility-settings
- PUT /api/accessibility-settings

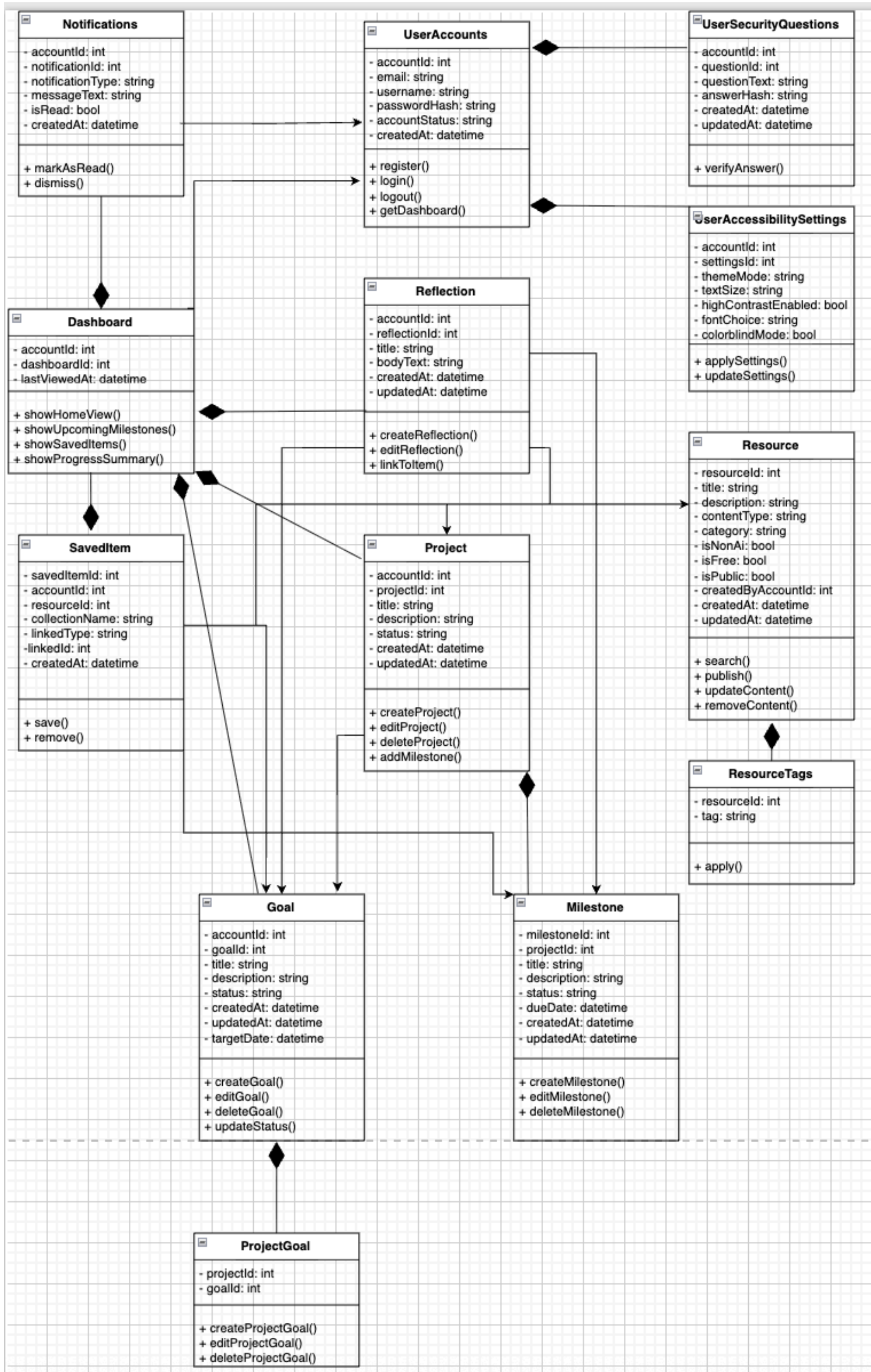
Significant non-trivial algorithm or process

Search ordering

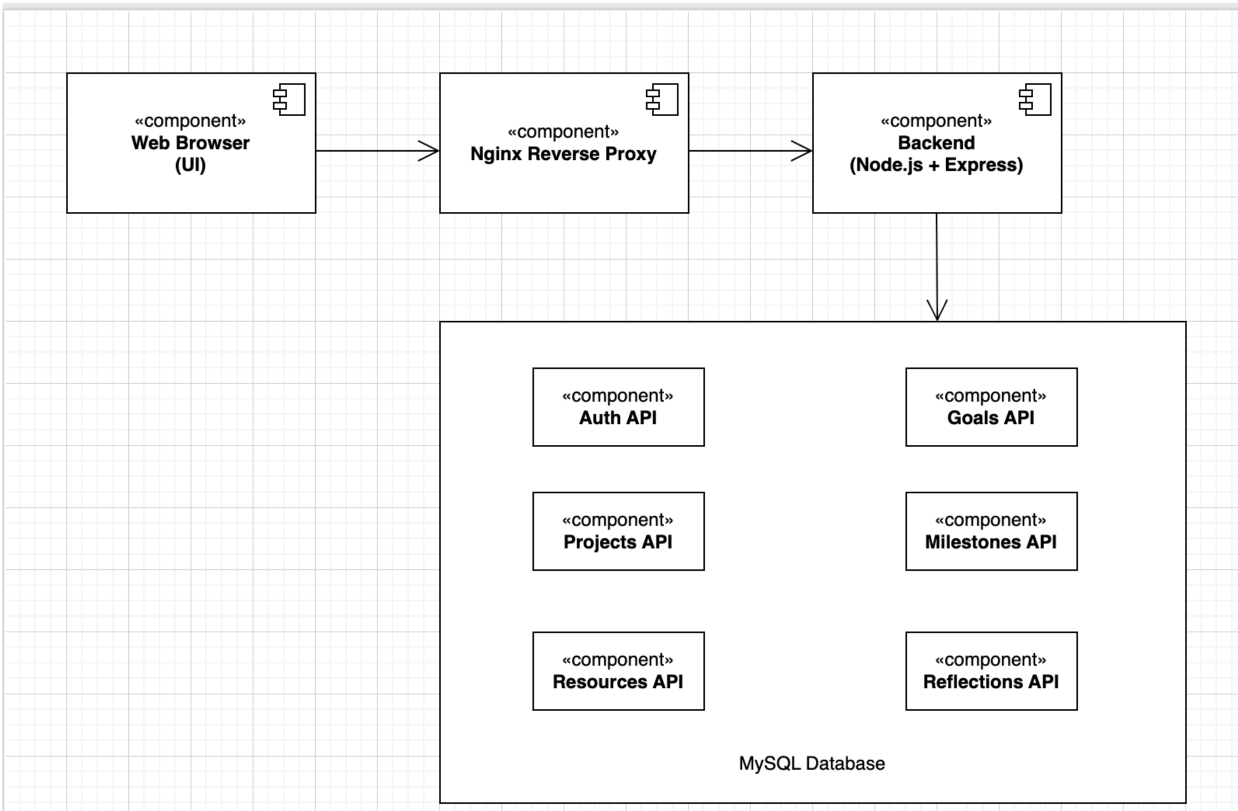
- The default order is newest first.
- If relevance sorting is enabled, a simple relevance score is used.
    - Title match ranks above description match.
    - Exact phrase match ranks above partial match.

4. High-Level UML Diagrams

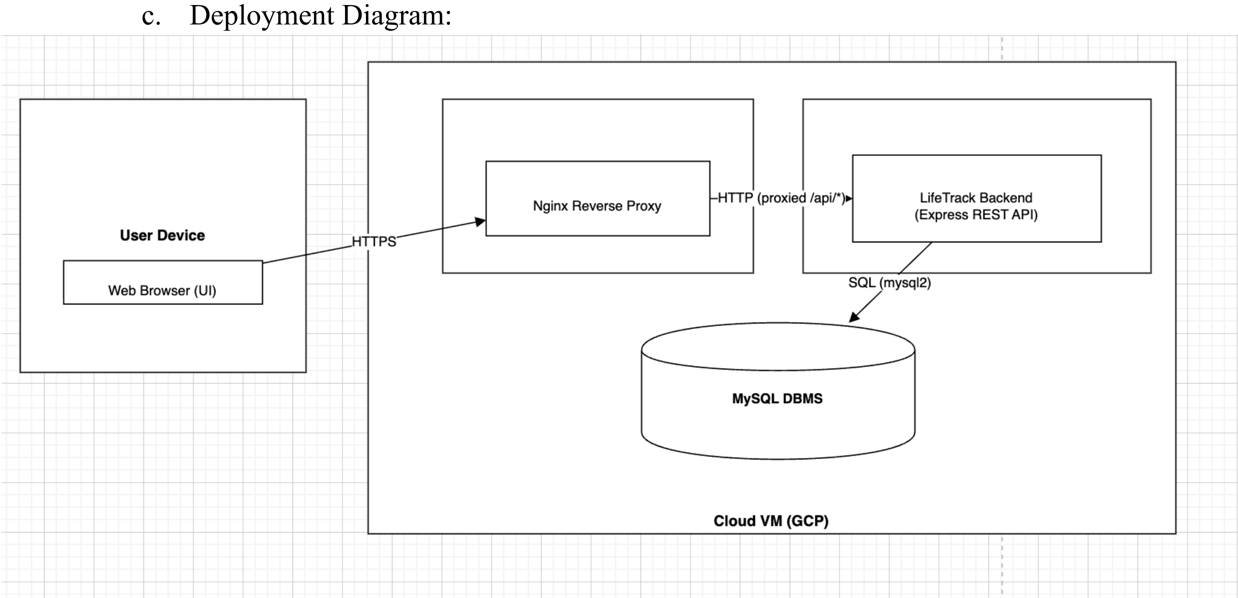
a. Class Diagram:



b. Component Diagram:



c. Deployment Diagram:



5. Identify actual key risks for your project at this time

Skill risks
While the team has strong technical development skills, we have limited experience with JavaScript. There is also limited experience in professional UI/UX design and creating visually polished, user-friendly interfaces. This could result in a functional but less engaging user experience that does not meet modern design standards.

Plan: All team members are expected to learn basic syntax in their free time and be familiar with it. Front-end engineers will study Bootstrap 5.2 documentation and modern UI/UX patterns. The team will use existing student platform interfaces (Canvas, Google Classroom) as design references. We will conduct early user testing with classmates to gather feedback on usability and iterate on designs before investing heavily in implementation.

Schedule Risks
The advanced search and filtering system (by category, usecase, skill area, content type) is a core feature, but more complex than initially estimated. Implementing efficient multi-criteria filtering with good performance could take longer than our current M2 - M3 timeline allows, potentially delaying our features.

Plan: Prioritize search functionality in Milestone 2 and implement a basic version first (keyword search only), then incrementally add filters in Milestone 3. Use MySQL indexing on frequently queried columns (category, tags) to ensure performance. If time runs short, we will deliver 3-4 core filters rather than all planned filters, ensuring quality over quantity.

Technical Risks
Our production server is running on a Google Cloud free-tier e-micro instance (1vCIP, 1GB RAM), which has limited resources and recently experienced unexpected downtime during Milestone 0 grading. The server may crash or become unresponsive under load, especially during demonstrations or when the TA grades our milestones.

Plan: Configure PM2 with automatic restart on failure (pm2 startup and pm2 save already completed). Implement basic error handling and logging to catch crashes early. Reserve a static IP address to prevent IP changes on VM restarts. Monitor server's health weekly via SSh and PM2 logs. If stability issues persist, request instructor approval to upgrade to a slightly larger GCP instance with free-tier limits.

Teamwork Risks
With six teammates working on different components (frontend, backend, and database), there is a risk of uneven workload distributions, merge conflicts in Git, or team members blocking each other's progress because of not meeting micro deadlines or technical difficulties.

Plan: At the start of this milestone, team lead will assign role responsibilities and task assignments at the start of each milestone. The team lead will give micro deadlines, and the team will complete tasks within the time frame to keep the project on track. We will meet during class and discuss responsibilities, deadlines, and any questions. There are also scheduled out-of-class meetings to make sure all engineers are on the same page and to prevent confusion as to what is happening in our code base. To prevent merge conflicts, GitHub master will make sure that pr are required and ensure there are no conflicts before code is merged. If a team member falls behind, it is their responsibility to tell the team a few days in advance of the deadlines, and there will be redistributions of tasks to unblock progress.

Legal Content Risks
The platform requires media-rich content, including images for resources, campus services, and user profiles. Using copyrighted images from the internet without permission could violate intellectual property laws and result in grade deductions or legal issues.

Plan: We plan to use only royalty-free images from sources like Unsplash, Pexels, or Pixabay with proper attribution. Create our own content (screenshots, diagrams) where possible. For team members' profile photos, use photos taken by team members or stock photos. Document images sources in the credits.MD files in the repo. Review all content before final submission to ensure no copyright violations.

6. Project Management

For the milestone 2 document, we have assigned each section to a team member. The team lead thought it was best for engineers to work on a section that is cohesive with their role. I have assigned the database engineer to work on “high-level, database organization”. Both front-end engineers will be working on UI mockups and storyboards (high-level only). Back-end engineers will be working on functional requirements and high-level UML diagrams. The team lead will be working on keyrocks and project management sections of the milestone 2 document.

The team lead has set a micro-deadline of March 2nd, a week from the assigned date, to finish their section of the document. The reasoning is that all engineers can finish the document and use the time up until March 17th to complete their vertical prototype. Once our document is finished, backend engineers and the database engineer will begin to work on the vertical prototype for several days. After they have completed their tasks, we plan to hold an outside-of-class meeting on March 6th. In this meeting, all engineers will be there to discuss what has been implemented. We will also discuss what other tasks there are to do and the micro deadlines for completion.

Currently, we are using Discord to communicate. We have organized channels for updates, deadlines, resources, and takeaways. Usually, at the start of milestones, we assign role responsibilities and post them on Discord. There are micro deadlines before the actual milestone due date to ensure we finish early and are able to revise our work, which are posted on the deadlines channel. After tasks and deadlines are assigned, the teammates work for several days, and then we have out-of-class meetings on Zoom to ensure there is no confusion and discuss what has been implemented so far. There is an expectation that each team member should always be communicating updates on what they have done, when they have finished their tasks, their progress, and any questions. We have a rule that you have to tell the team if you need more time to complete tasks, so we can help out or figure out a way to make implementation quicker. We always make sure to finish all of our work a day or two before the actual milestone deadline so we can do quality checks over our work.

As of now, the work coordination and team work has been excellent. We have been hitting all milestone deadlines on the team, including our own micro deadlines. Eventually, we will move to Trello for better project management.